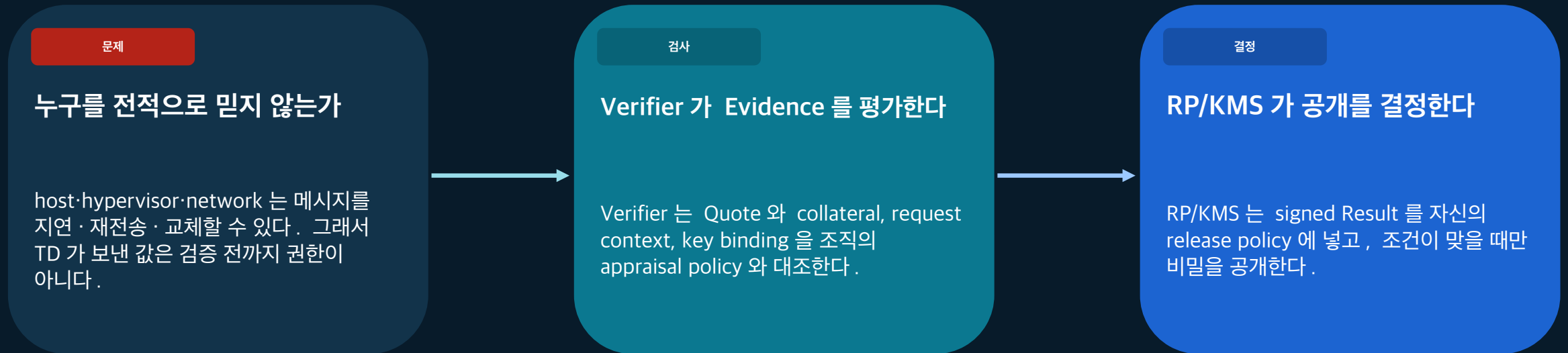


Intel TDX 원격 증명은 승인한 VM에만 비밀을 공개하는 통제다

01

클라우드 운영자를 전적으로 믿지 않아도, hardware Evidence 와 조직 policy 가 함께 secret release 를 조건화한다.



읽는 문장

즉, Quote 가 유효하다는 사실은 Verifier 의 appraisal 결과일 뿐이다. 비밀을 공개할 권한은 Result 를 검증한 RP/KMS 의 별도 정책에서 나온다.

같은 경로에 있어도 같은 trust role 은 아니다

02

역할을 분리하면 “누가 무엇을 만들고, 누가 무엇을 믿으며, 누가 행동을 결정하는지”가 보인다.

TD

TDX 가 보호하는 VM

임시 키를 만들고 TDREPORT 를 요청한다.

QGS daemon

untrusted host transport

TDREPORT·Quote 를 운반하지만 accepted Quote 를 서명하지 못한다.

TDQE

Quoting Enclave

같은 플랫폼 TDREPORT 를 확인하고 TD Quote 에 서명한다.

Verifier

증거 평가자

nonce 를 내부 보관하고 Evidence 를 평가해 Result 를 서명한다.

RP/KMS

공개 결정자

Result 를 확인하고 release policy 를 집행한다.

Intel PCS

서명된 collateral 의 권위 원천

PCK·CRL·QE identity·TCB info 의 signed origin 이다.

signed collateral



PCCS

collateral cache / transport

PCS 의 signed bytes 를 운반하지만 자체 권위는 없다.

구분 규칙

QGS 가 Quote generation 을 조율해도 signer 는 TDQE 다. PCS 는 signed collateral 의 권위 원천이고 PCCS 는 cache/transport 다. Verifier 가 Result 를 서명해도 secret 을 공개하는 owner 는 RP/KMS 다.

공격 가능한 경로는 Evidence 를 운반할 수 있어도 권한을 만들 수는 없다

03

host·network·QGS·PCCS·Attester input 을 검증 전까지 신뢰하지 않는 위협 모델에서 시작한다 .

공격자가 할 수 있는 일

drop · delay · replay · reorder · substitute · deny service

신뢰하는 기반

CPU/TDX Module·TDQE/PCK chain·PCS signatures·Verifier policy/key·RP/KMS policy/time·선택한 암호 구현·TD 안의 임시 private key custody

보장하지 않는 것

availability · code correctness · future behavior · 모든 side-channel · protected external I/O

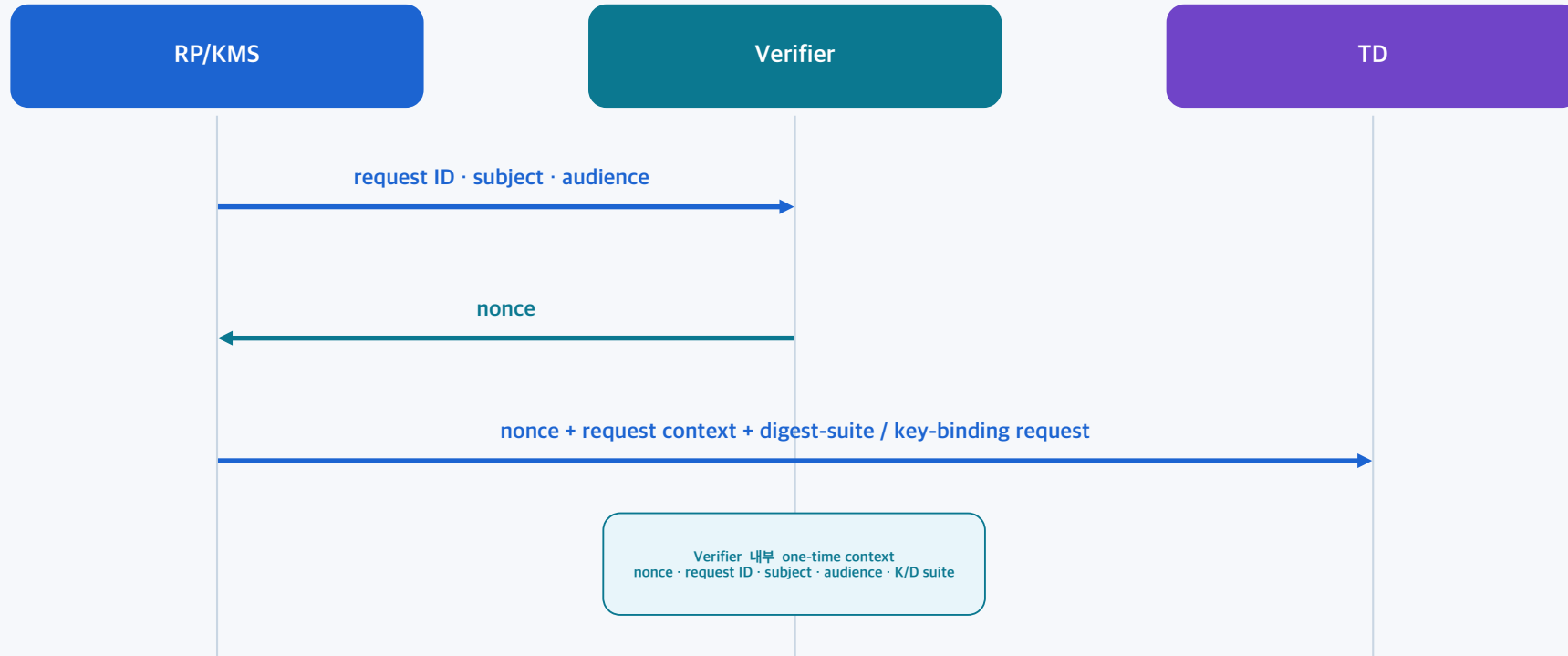
경계의 의미

untrusted path 가 Quote 를 재전송하거나 바뀌도 Verifier 가 retained nonce·request·key binding 을 다시 비교한다 . 비교 실패 , stale Evidence, 또는 검증 경로 장애에서는 secret release 를 NO KEY 로 종료한다 . degraded mode 가 필요하면 자산 공개가 없는 별도 정책을 미리 정한다 .

nonce 는 “이 Quote 가 지금 이 요청을 위한 것인가”를 확인하는 일회용 기준이다

04

Verifier 가 기대 문맥을 보관하고 , RP/KMS 가 그 문맥을 TD 까지 전달해야 replay 와 substitution 을 거부할 수 있다 .



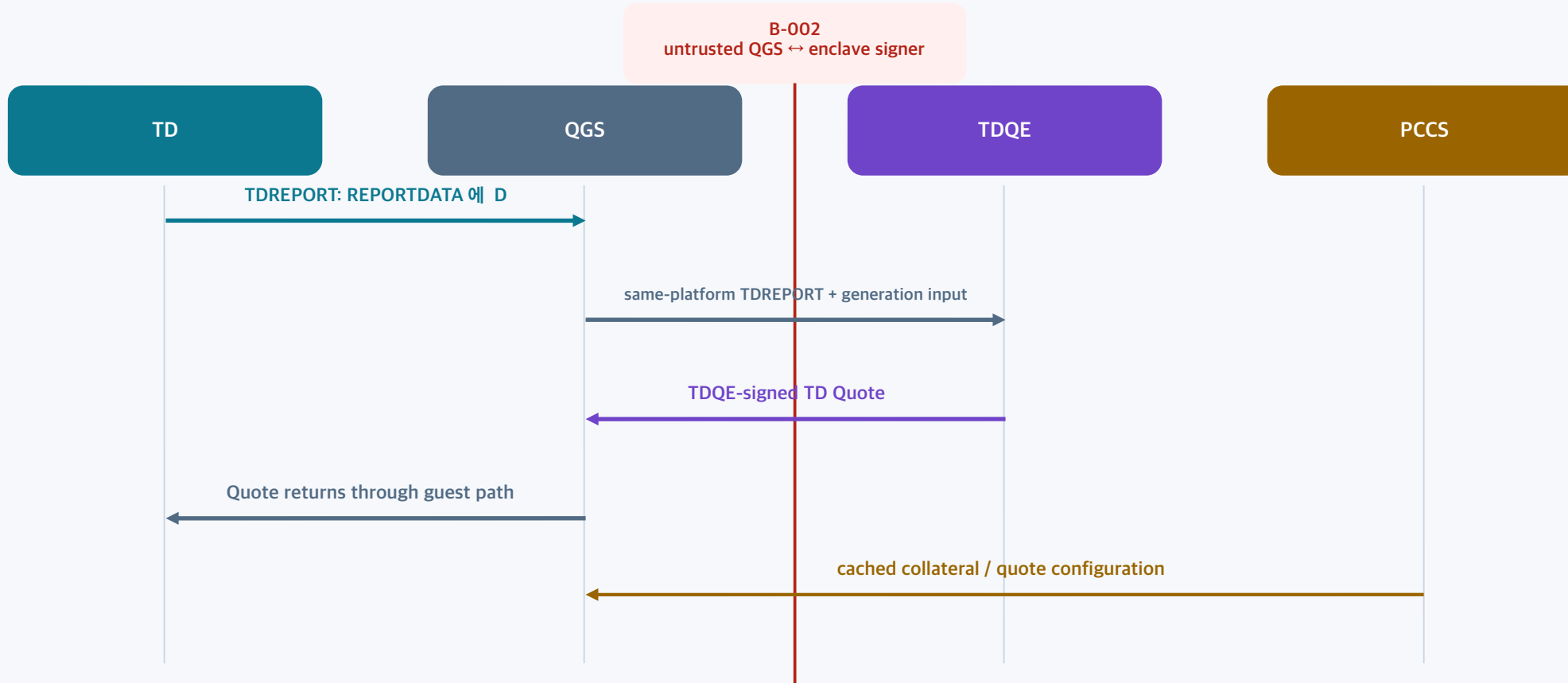
읽는 문장

Verifier 는 nonce 와 full expected context 를 내부 상태로 terminal event 까지 보관한다 . TD 는 nonce·request ID·subject·audience·K 를 D 에 묶고 , Verifier 는 key bytes 로 같은 D 를 재계산한다 . 재시도는 기존 record 를 닫고 새 nonce 로 시작한다 .

QGS 는 운반하고 TDQE 가 서명한다

05

Quote generation path 에서 transport 성공과 signing authority 를 분리해야 host-side daemon 을 과신하지 않는다 .



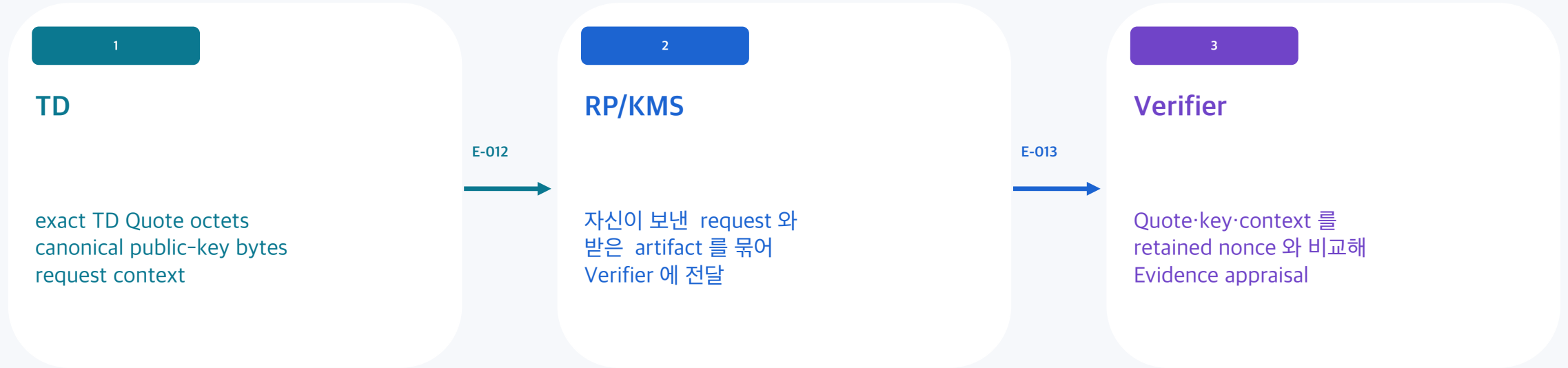
읽는 문장

TD 는 TDREPORT 를 QGS 에 넘기지만 , QGS 가 받은 report 에 신뢰를 더하지는 않는다 . TDQE 가 같은 platform report 를 확인하고 Quote 를 서명하며 , Verifier 는 나중에 그 Quote 와 collateral 을 별도로 검증한다 .

TDQE 가 서명해 TD 로 반환된 Quote 와 key bytes 는 RP/KMS 를 거쳐 Verifier 의 appraisal 로 들어간다

06

이 transport route 를 분명히 하면 QGS 의 guest path 와 Evidence delivery path 를 혼동하지 않는다 .



E-010→E-013

TDQE 가 TD Quote 를 생성 · 서명해 QGS 를 거쳐 TD 로 돌려준다 . TD 는 canonical key bytes 와 identical request context 를 붙여 RP/KMS 에 보내고 , RP/KMS 가 이를 Verifier 에 전달한다 . Verifier 는 retained expected context 와 비교해 Evidence 가 이 request 와 이 key 를 위한 것인지 판단한다 .

K, D, E 는 “무엇을 확인했는가”를 세 종류의 bytes 에 묶는다

07

아래 식은 symbolic invariant 다 . 배포 전에는 모든 byte-level choice 를 profile 로 고정해야 하며 , 이 자체가 RFC wire format 은 아니다 .

$$K = \text{Hash}(\text{"tdx-ra/key/v1"} \parallel \text{key_algorithm} \parallel \text{canonical_key_bytes})$$
$$D = \text{Hash}(\text{"tdx-ra/context/v1"} \parallel \text{canonical_encode}(\text{nonce, request_id, subject, audience, K}))$$
$$E = \text{Hash}(\text{"tdx-ra/evidence/v1"} \parallel \text{exact_TD_Quote_octets})$$

K

승인할 공개키의 digest

protocol-canonical public-key bytes 와 key algorithm 을 묶어 , 어느 키가 나중에 secret 을 받을 수 있는지 정한다 .

D

요청 문맥과 K 의 digest

nonce·request ID·subject·audience·K 를 묶어 fixed 64-byte REPORTDATA mapping 에 넣는다 .

E

정확한 Quote 원문의 digest

Result 가 어떤 transported Quote Evidence 를 평가한 결과인지 RP/KMS 가 확인하게 한다 .

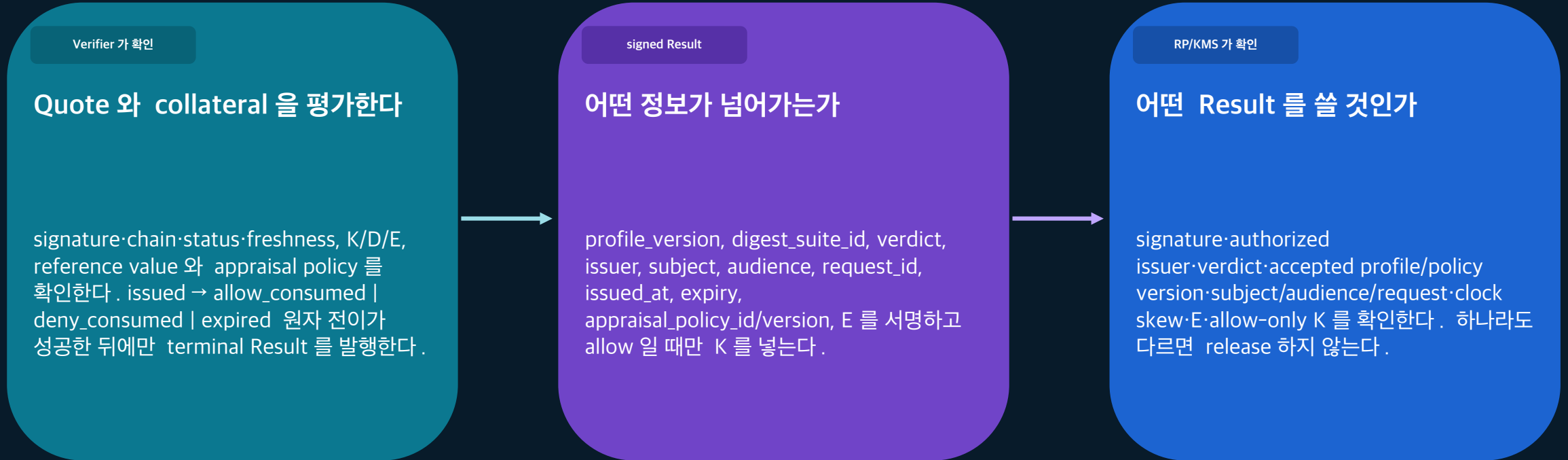
비교 규칙 · 한정자

Verifier 는 retained nonce 와 받은 context·key bytes 로 K 와 D 를 재계산해 REPORTDATA 와 비교하고 , exact Quote bytes 로 E 를 계산한다 . 배포 profile 은 hash/output length, 허용 key algorithm·canonical encoding, field order·length delimiting, domain tag/version, D 의 64-byte REPORTDATA mapping 을 확정해야 한다 .

Verifier Result 는 평가의 서명된 기록이고 , 비밀 공개 명령은 아니다

08

Result 에는 profile·digest suite· 평가 정책 ·E 와 allow-only K 가 들어가 RP/KMS 가 무엇을 승인하는지 다시 확인할 수 있어야 한다 .



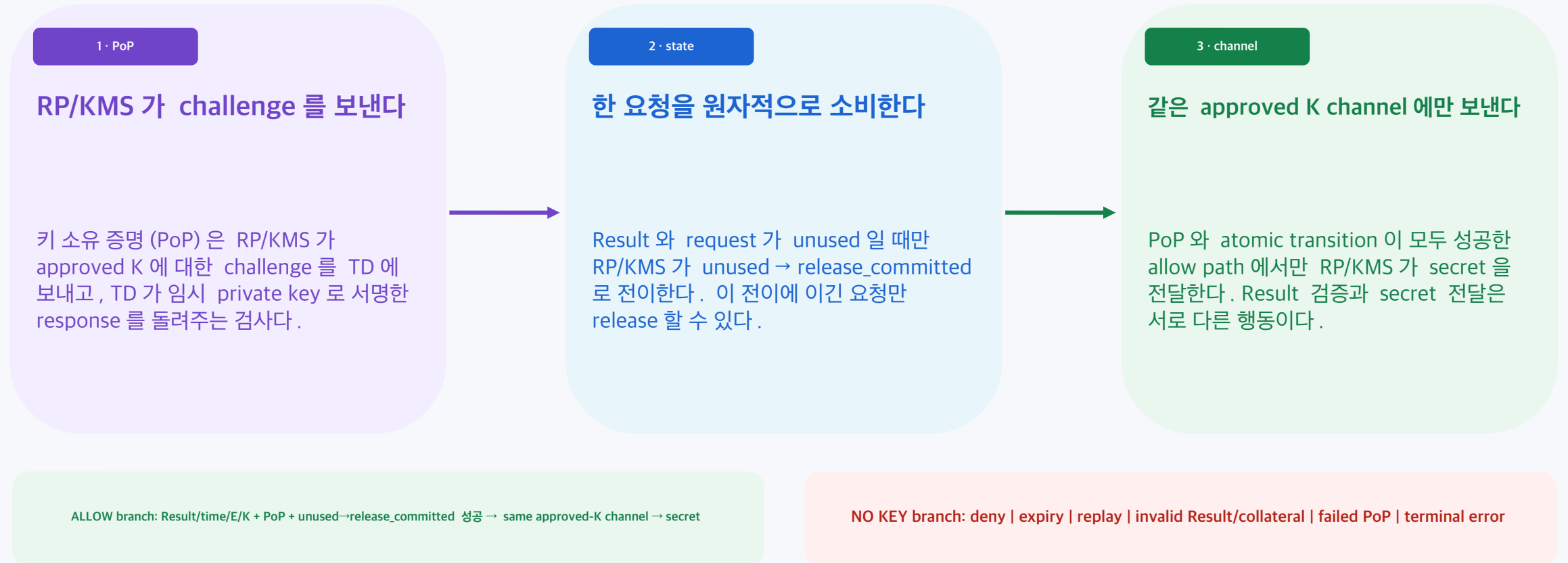
읽는 문장

Verifier 가 signed Result 를 RP/KMS 에 보낸다 . RP/KMS 가 Result signature·accepted profile/policy·time·E/K 를 확인해도 그 자체는 secret release 가 아니며 , 다음 키 소유 증명 (PoP) 과 replay-state 전이가 별도로 성공해야 한다 .

비밀 공개는 키 소유 증명 (PoP) 과 일회용 state 가 통과한 뒤 한 번만 수행한다

09

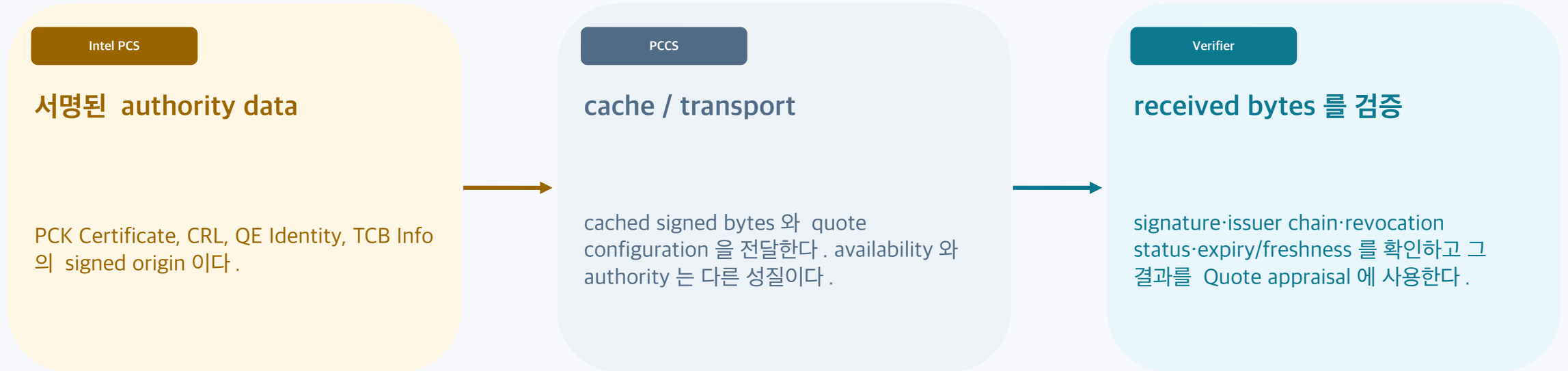
Result 가 allow 여도 RP/KMS 는 approved K 의 private key 를 실제로 가진 TD 인지 확인하고 replay 를 막아야 한다 .



PCCS 가 살아 있다는 사실이 collateral 이 유효하다는 뜻은 아니다

10

PCS 는 signed authority data 의 원천이고 PCCS 는 cache/transport 이므로 , Verifier 는 받은 bytes 자체를 검증한다 .



운영 결과

PCCS outage, nextUpdate 실패 , revocation, TCB 변경은 운영 policy 가 처리해야 한다 . Verifier 는 PCCS cache 를 믿어서가 아니라 received collateral 의 cryptographic validity 와 freshness 를 확인해서만 사용한다 .

전체 Quote→Result 경로는 reported TD state 를 확인하지만 workload 의 모든 성질을 증명하지는 않는다

11

proof boundary 를 분리해야 valid Quote 와 signed Result 를 code correctness 나 container identity 로 과대 해석하지 않는다 .

이 경로로 알 수 있는 것

- Quote integrity 와 accepted quoting credential 연결
- reported TD measurement·attribute·TCB state
- D 가 retained context 와 canonical key 에 묶였는지
- reference value 와 비교 가능한지
- Result 가 E·policy version·allow-only K 를 가리키는지
- 별도 Result·PoP·authorization 검사를 거친 release path

Quote 만으로는 알 수 없는 것

- code correctness 또는 vulnerability absence
- TD 안의 container/process 자동 identity
- attestation 이후의 future-safe behavior
- retained·terminal challenge 없이 Evidence freshness
- 별도 검사 없는 private-key possession·Result authenticity·최종 authorization
- availability 와 모든 side-channel·I/O·physical·supply-chain risk 제거

설계 판단

Quote/Result 는 workload correctness·container identity 를 증명하지 않는다 . release 조건이면 workload Evidence 와 runtime control 을 추가하고 , freshness·PoP·authorization 은 각각 독립 검사한다 .

Intel-rats 는 판단 구조를 배우는 학습 자료이지 운영 verifier 기준은 아니다

12

프로젝트는 도메인 판단 구조와 설계 검토 질문을 보이게 하지만 production implementation 을 보증하지는 않는다 .

취할 강점

도메인 구조를 보이게 한다

TD·Guest Agent·Verifier·Key Broker·PCS/PCCS·policy owner 를 하나의 release decision 으로 연결해 design review 질문을 만든다 .

RFC 와 다른 부분

표준 Background Check 와 route 가 다르다

RFC 9334 Background Check 은 Attester→RP→Verifier 다 . Intel-rats 는 Evidence 선등록 후 RP 조회를 쓰며 aggregate direct edge 와 상세 Guest Agent route 가 섞인다 .

구현 한계

운영 구현이 빠져 있다

nonce·audience·expiry·workload Evidence 가 first-class type 이 아니고 concrete Quote/DCAP API/ 배포 코드 / 정책 언어가 없다 . deny-without-key 는 교육용 불변조건이지 runtime enforcement 보증이 아니다 .

reader question

Intel-rats 는 “누가 Evidence 를 평가하고 , 누가 release policy 를 소유하며 , 어떤 binding 과 failure state 가 explicit 한가”를 묻는 학습 도구다 . 실제 deployment 는 RFC 흐름 · 구현 API· 정책 언어 · 운영 enforcement 를 별도로 확정해야 한다 .

TDX Quote 를 container identity 로 읽으면 증명 경계를 넘어선다

13

SGX 와 TDX 는 Evidence → appraisal → policy action 구조를 공유하지만 , attested scope 와 concrete mechanism 은 다르다 .

SGX

enclave identity 중심

SGX 는 MRENCLAVE·MRSIGNER 같은 enclave identity 를 중심으로 attestation 을 읽는다 . 그래도 code correctness 나 future safety 는 별도 문제다 .

TDX

whole Trust Domain / VM 경계 중심

TDX Quote 는 firmware·OS·workload 를 포함할 수 있는 TD 전체 VM state 를 보고한다 . TD 안의 개별 container 신원은 자동으로 나오지 않는다 .

scope limit

컨테이너가 준 값을 REPORTDATA 에 넣어도 컨테이너의 독립 identity 가 증명되는 것은 아니다 . protected identity 가 container 라면 workload-specific Evidence 와 policy check 를 별도로 설계한다 .

첫 파일럿은 회수 가능한 단일 key-release path 와 명확한 책임자에서 시작한다

14

출시 전에는 appraisal 과 release 의 책임자 , collateral 운영 , key binding, replay/expiry failure 를 수치 기준으로 확정한다 .

보안 · 플랫폼 책임자

reference values, TCB/freshness/binding appraisal policy, revocation/re-attestation 기준

보호 자산 서비스 책임자

Result-use/release policy, subject/audience/clock skew, fail-closed 운영 조건

플랫폼 운영

PCS/PCCS 갱신 , collateral outage posture, revocation-to-deny SLO, audit/recovery

개발자 · 아키텍트

challenge state machine, K/D/E suite, Result/PoP, negative tests, key rotation

출시 gate: 단일 revocable key-release path 는 다섯 조건이 모두 달힌 뒤에만 연다 .

① terminal 1 회

allow·deny·expiry
요청은 한 번만 소비

② numeric SLO

outage·skew·revoke
re-attestation

③ policy audit

version + approver
감사 가능

④ key rotation

revoke·rollback
리허설

⑤ 공개 key 0

forged·replay·substitute
Verifier/PCCS failure

Appendix: 10 개 Point claim 이 독자 문장과 어느 slide 에서 만나는가

15

coverage 는 ID footer 가 아니라 실제 Korean assertion·narrative·proof limit 으로 확인한다 . 자세한 locator 는 final/prose-trace.json 과 structural-coverage-map.json 에 남긴다 .

C-001

Evidence 평가는 비밀 공개 권한이 아니다

slide 01

C-006

K/D/E 와 Result profile 은 구현 전 version-fixed profile 이 필요하다

slide 07-09

C-002

보호 · 운반 · 서명 · 평가 · 공개는 다른 역할이다

slide 02, 05

C-007

Quote/Result 는 security proof 의 일부이지 모든 property 의 증명은 아니다

slide 11

C-003

untrusted path 는 권한을 만들 수 없고 NO KEY 로 끝난다

slide 03, 04

C-008

Intel-rats 는 학습 자료이며 운영 implementation 기준은 아니다

slide 12

C-004

request·Quote·Result·PoP·same-key release 는 닫힌 경로다

slide 04-09

C-009

TDX Quote 는 TD/VM scope 이지 container identity 가 아니다

slide 13

C-005

PCS authority 와 PCCS cache 는 다르다

slide 10

C-010

단일 revocable pilot 과 책임 ·SLO·negative gate 가 먼저다

slide 14

audit locator

이 Appendix 는 claim-level reader-visible trace 다 . model node/edge/boundary/context/state/proof/non-proof/implication/caveat/source marker 의 exact location 은 package JSON 에서 재검증할 수 있다 .

Appendix: canonical Point hash 와 [S-...] source marker 를 독자에게 보이게 한다

16

slide footer 의 [S-...]는 passed Point 의 source_map stable reference 다 . 이 package 에는 bibliography title registry 가 없으므로 , 존재하지 않는 서지를 추정해 쓰지 않는다 .

Canonical Point SHA-256 59bb8e0d71c321cc18cb4880e27dbbe7966511257a4d4c33e7e7a538fbc38891

C-001 S-002 · S-003 · S-007

C-002 S-002 · S-003 · S-005 · S-007 · S-009

C-003 S-002 · S-003 · S-005 · S-009

C-004 S-002 · S-003 · S-004 · S-005 · S-006 · S-008 · S-010 · S-011

C-005 S-003 · S-005

C-006 S-002 · S-004 · S-005 · S-011

C-007 S-002 · S-003 · S-009

C-008 S-001 · S-002 · S-010

C-009 S-001 · S-002 · S-004 · S-010 · S-012

C-010 S-002 · S-003 · S-007 · S-010 · S-011

source audit rule

근거를 다시 확인할 때는 source marker → claim → reader-visible slide 문장 → canonical Point 를 역방향으로 추적한다 . full source citation 이 필요한 delivery 에는 source registry 를 canonical Point package 에 추가한 뒤 deck 을 재생성한다 .